the whole notebook, one note per page

Margin.Notes

Distributed design patterns, explained with problems, analogies, doodles, trade-offs, ratings and pitfalls.

01 **Circuit Breaker** — *resilience*

Fail fast, fall back, then test carefully.

02 **Saga** — *consistency*

Local steps plus compensating "undo" steps.

a.k.a. "stop calling the broken thing"

Circuit Breaker

TL;DR

Wrap every call to a remote dependency in a little switch. When too many calls fail, the switch **trips open** and later calls fail *instantly* with a fallback, instead of waiting. After a cool-down it lets a few test calls through. If they work, it **closes** again.

Family: *resilience / stability* | **Lives in:** *the caller (client, gateway or service mesh)*

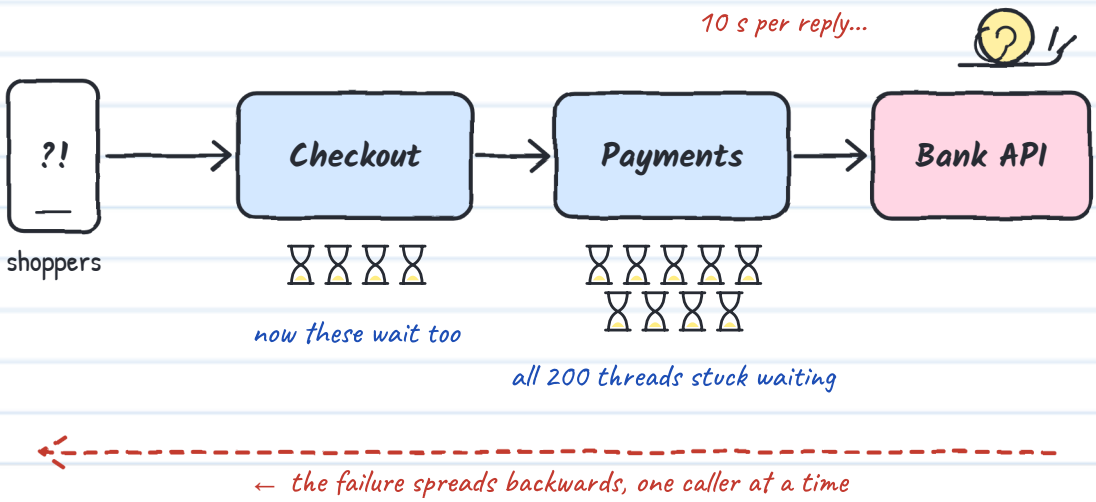
Popularised by: *Michael Nygard, Release It! (2007)* | **Pairs with:** *timeouts, retries, bulkheads*

1 The problem *why do we even need this?*

In a distributed system, services call each other over the network. Sooner or later one of them gets slow or stops answering. The scary part is what that does to everyone who calls it.

Say **Checkout** calls **Payments**, and Payments calls the **Bank API**. The bank has a bad day and every reply takes 10 seconds. Nothing has "crashed", but:

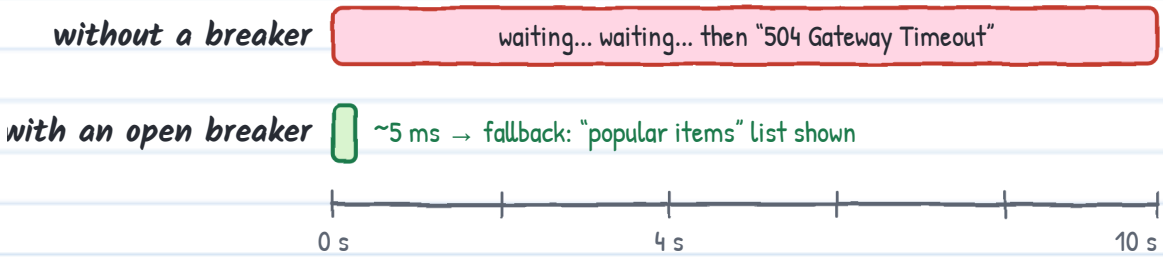
1. Each Payments request now holds a thread (and a DB connection, and memory) for 10 s while it waits.
2. Payments has a pool of 200 threads. New requests queue up behind the stuck ones.
3. Checkout's calls to Payments now also wait, so *Checkout's* threads fill up too.
4. Shoppers see spinners, hit refresh, and send *even more* requests. Retries pile on.
5. The whole site is down because one outside API got slow. This is a **cascading failure**.



▲ *Nothing is "down". One dependency is just slow, and the waiting spreads upstream.*

> ↳ *quick math from class: 50 orders/s × 10 s wait = 500 calls in flight, but only 200 threads → the pool is full after about 4 seconds!*

Key insight: a slow dependency is more dangerous than a dead one. A dead one fails fast ("connection refused"). A slow one quietly holds your resources hostage until the timeout.

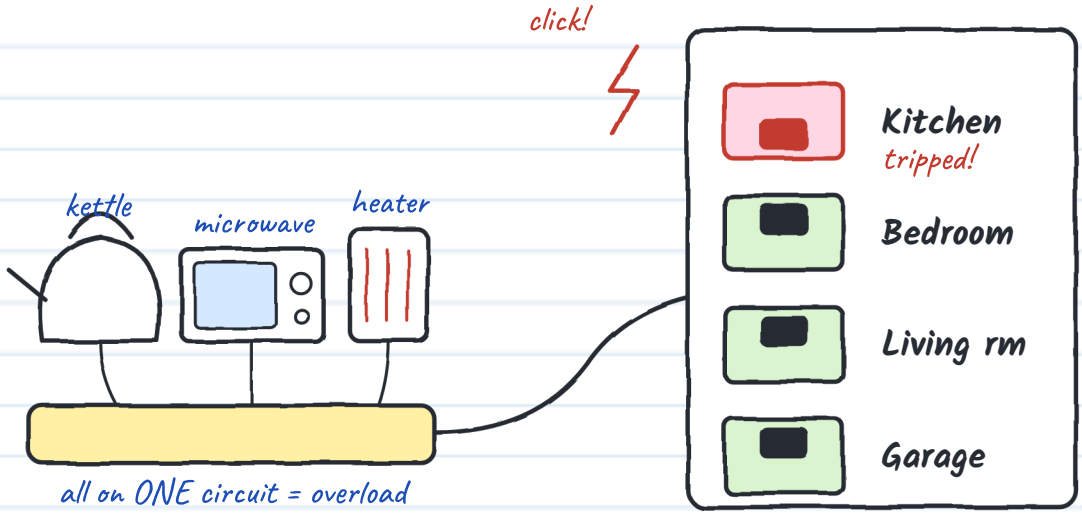


▲ *Failing fast frees the thread right away and the shopper still sees something useful.*

2 Real-life analogy *the breaker panel at home*

The name comes straight from the electrical panel in your house. Plug a kettle, a microwave and a heater into one kitchen circuit and switch them all on. The current spikes, the wire starts to heat up, and **click**: the kitchen breaker trips.

- The kitchen goes dark, but the **house doesn't catch fire** and the bedroom lights stay on. The damage is contained.
- You don't flip it back on every two seconds. You **wait**, unplug the heater, then flip it **once** to test.
- If it trips again right away, something is still wrong, so you leave it off longer.



▲ One circuit trips so the rest of the house keeps its power. Same idea for services.

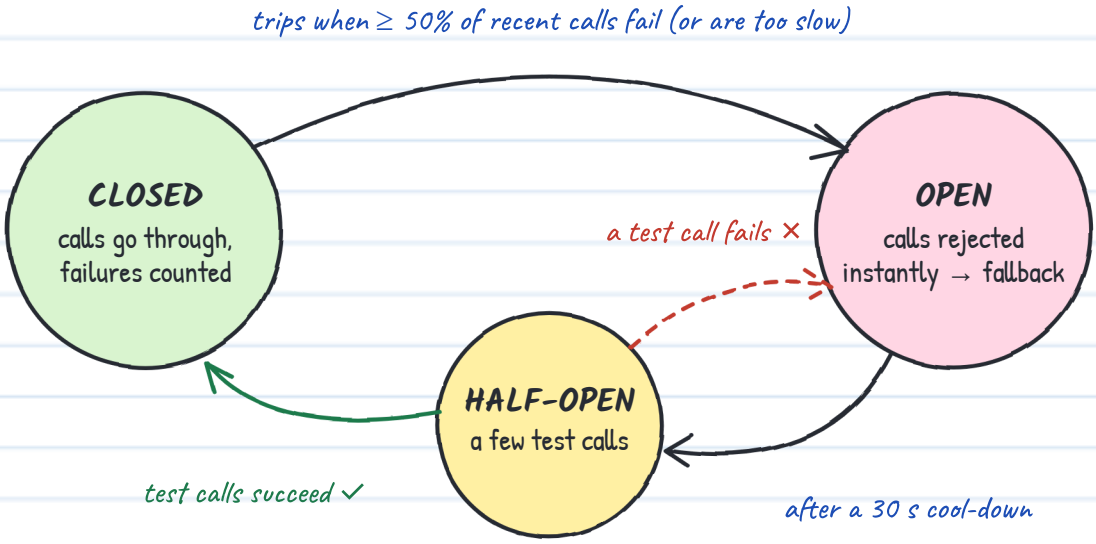
Translating the house to software

At home	In your system
Too much current on one circuit →	Too many failed or slow calls to one dependency
The breaker trips (switch goes down) →	The breaker goes **OPEN** and stops sending calls
Kitchen is dark, the rest of the house is fine →	One feature is degraded, the rest of the app keeps working
Use a camping lamp in the kitchen for now →	**Fallback**: cached data, a default, or a friendly message
Wait a bit, then flip it once to test →	**HALF-OPEN**: after a cool-down, let a few test calls through
It stays on, so you go back to normal →	Test calls succeed, so the breaker goes back to **CLOSED**

> ↳ another way to remember it: a friend who never picks up. After 5 missed calls you stop calling for an hour, then send ONE text to check. You don't ring 50 times in a row.

3 How it solves it *three states*

The breaker sits between the caller and the dependency and watches the results of recent calls. It's a small **state machine**:



▲ *Green = normal. Pink = protecting everyone. Yellow = carefully testing the water.*

1 CLOSED (normal)

Every call passes through. The breaker records each result in a **sliding window**, for example the last 20 calls or the last 10 seconds. Timeouts, connection errors and 5xx responses count as failures. Calls slower than a threshold (say 2 s) can count as "slow calls".

2 Trip → OPEN

Once the window has enough calls (a **minimum** like 20, so 2 failures at 3 a.m. don't trip it) and the failure rate or slow-call rate crosses the threshold (say 50%), the breaker opens.

3 OPEN (fail fast)

Calls don't touch the network at all. The breaker throws a "circuit open" error right away, and the caller runs its **fallback**. The dependency gets breathing room to recover instead of being hammered.

4 Cool-down → HALF-OPEN

After a wait (say 30 s) the breaker lets a small number of **trial calls** through (say 3). Everything else is still rejected.

5 Decide

If the trial calls succeed, go back to CLOSED and reset the counters. If any of them fails, go straight back to OPEN and start the cool-down again. Some libraries make each new cool-down longer (exponential back-off).

What can a fallback be?

Cached / stale data

Show yesterday's exchange rate or the last known stock level, with a "may be out of date" note.

A sensible default

Recommendations down? Show the bestsellers list. Avatar service down? Show initials.

Do it later

Put the request on a queue ("we'll email your receipt") and process it when the dependency is back.

Fail clearly

If nothing sensible exists, return a fast, honest error (503 + `Retry-After`) instead of a 30 s hang.

Vocabulary for the exam

Failure-rate threshold

% of failed calls in the window that trips the breaker (often 50%).

Slow-call threshold

A call slower than this counts as "slow". Too many slow calls also trip it.

Sliding window

The recent calls the breaker judges by: count-based (last N) or time-based (last N seconds).

Minimum calls

Don't decide anything until the window holds at least this many calls.

Wait duration (open)

How long to stay OPEN before testing again.

Permitted calls (half-open)

How many trial calls to let through when testing.

circuit-breaker.ts — *simplified, count-based window*

```ts
type State = "CLOSED" | "OPEN" | "HALF_OPEN";

export class CircuitBreaker {
  private state: State = "CLOSED";
  private results: boolean[] = [];    // true = failure
  private openedAt = 0;
  private trials = 0;

  constructor(private opts = {
    windowSize: 20, minCalls: 20, failureRate: 0.5,
    openMs: 30_000, trialCalls: 3,
  }) {}

  async call<T>(fn: () => Promise<T>, fallback: () => T): Promise<T> {
    if (this.state === "OPEN") {
      if (Date.now() - this.openedAt < this.opts.openMs) return fallback(); // fail fast
      this.state = "HALF_OPEN"; this.trials = 0;
    }
    if (this.state === "HALF_OPEN" && this.trials++ >= this.opts.trialCalls) return fallback();
    try {
      const value = await fn();          // fn MUST have its own timeout
      this.record(false);
      return value;
    } catch (err) {
      if (!isRemoteFailure(err)) throw err;  // 400/404 are not the dependency's fault
      this.record(true);
      return fallback();
    }
  }

  private record(failed: boolean) {
    if (this.state === "HALF_OPEN") {
      if (failed) return this.trip();
      if (this.trials >= this.opts.trialCalls) { this.state = "CLOSED"; this.results = []; }
      return;
    }
    this.results = [...this.results, failed].slice(-this.opts.windowSize);
    const fails = this.results.filter(Boolean).length;
    if (this.results.length >= this.opts.minCalls &&
```

```
        fails / this.results.length >= this.opts.failureRate) this.trip();
    }

    private trip() { this.state = "OPEN"; this.openedAt = Date.now(); log("breaker
OPEN"); }
}
```

↳ *in real projects don't hand-roll this. Use Resilience4j (Java), Polly (.NET), opossum (Node), gobreaker (Go), or let the service mesh do it (Envoy / Istio "outlier detection").*

4 Gains vs losses *nothing is free*

What we gain

+ **Stops cascading failures.** One bad dependency can't take the whole system down.
+ **Fails fast.** Answers in milliseconds instead of hanging for a timeout, so threads and connections are freed.
+ **Gives the sick service room to recover** instead of hammering it with traffic and retries.
+ **Graceful degradation.** Users get a fallback instead of an error page.
+ **A clear health signal.** "Breaker opened" is an easy thing to alert on and graph.

What we lose

− **More things to tune.** Thresholds too tight and it flaps open on a blip; too loose and it never trips.
− **Degraded answers.** While OPEN, users get stale or partial data, which must be acceptable for the business.
− **Inconsistent views.** 20 instances = 20 separate breakers, each tripping at a different moment.
− **Harder testing.** You need fault injection to prove each state and the fallback actually work.
− **Can hide problems** if nobody watches it. The fallback looks "fine" while the dependency burns.

5 Architecture ratings *1 = poor · 5 = great*

Same nine characteristics we use for architecture styles. Each line compares a typical service **without** a breaker to the same service **with** one.

Characteristic	With the pattern	Before → after	Why
Simplicity *easy to understand?*		4 → 3 ▼ −1	One more moving part, three states and several settings to understand.

Characteristic	With the pattern	Before → after	Why
Low cost *cheap to build & run?*		4 → 4 = no change	Free libraries, tiny CPU overhead. Mostly costs tuning time.
Deployability *safe, easy releases?*		4 → 4 = no change	No real change. Config can live in a file or the mesh.
Testability *test with confidence?*		4 → 3 ▼ −1	Each state and each fallback needs fault-injection tests.
Scalability *more users & data?*		3 → 3 = no change	Doesn't add capacity by itself. It protects the capacity you have.
Elasticity *sudden spikes?*		2 → 3 ▲ +1	Sheds load from an overloaded dependency, buying autoscaling some time.
Fault tolerance *failure stays contained?*		2 → 5 ▲ +3	The whole point. One slow dependency no longer sinks every caller.
Performance *fast typical request?*		3 → 4 ▲ +1	Normal path unchanged. During incidents, ms instead of a 10 s hang.
Evolvability *easy to change?*		4 → 4 = no change	Wraps a call site. Easy to add, remove or retune later.

kept *gained* *lost* *ratings are relative guidance for a typical system, not laws*

6 Common pitfalls *teacher said these show up in incidents a LOT*

✕ No timeout under the breaker

The breaker only counts a failure when a call ends. With a 60 s default timeout, it takes minutes to trip and threads are stuck the whole time. ***Fix*** → always pair it with a tight timeout on each call.

✕ One breaker for everything

A single breaker around the whole Payments client trips for *all* endpoints when only `/refunds` is broken. ***Fix*** → one breaker per dependency, and per endpoint when they fail independently.

✕ Counting the wrong errors

A burst of 400s or 404s from bad user input trips the breaker, even though the dependency is healthy. ***Fix*** → count only timeouts, connection errors, 5xx and 429.

✕ Deciding on tiny samples

At 3 a.m., 2 of 3 calls fail, that's 66%, and the breaker trips for nothing. ***Fix*** → set a minimum number of calls before the rate counts.

Retrying a "circuit open" error

A retry loop wraps the breaker and keeps retrying the instant rejection, burning CPU and filling logs. **Fix** → treat "breaker open" as non-retryable and go to the fallback.

A fallback that is just as fragile

The fallback calls *another* remote service that is overloaded too, so you've moved the problem. **Fix** → keep fallbacks local and cheap: cache, default, queue.

A silent breaker

It opens and closes all day and nobody notices, because the fallback hides it. **Fix** → log and emit metrics on every state change, and alert when it stays open.

7 When to use it (and when not)

Use it when...

- You make **synchronous** calls (HTTP, gRPC) to services you don't control or that can get slow.
- Calling third-party APIs: payment providers, maps, SMS, shipping rates.
- One page or request fans out to many services.
- A sensible fallback exists, or failing fast is better than waiting.

Skip it when...

- The call is in-process or to a local resource (a file, an in-memory cache).
- Work is async through a queue. Use retries with back-off and a dead-letter queue instead.
- You're using it to avoid fixing a dependency that is always broken.
- Traffic is so low that the window never fills enough to decide anything.

Related patterns

Timeouts & retries · the breaker's best friend

Bulkhead · separate thread pools per dependency

Rate limiter · protects the callee from you

Fallback / graceful degradation

Health checks & outlier detection

8 Self-check quiz

Q Why is a slow dependency worse than a dead one? ✓

A dead service fails fast (connection refused), so callers move on. A slow one holds every caller's threads and connections until the timeout, so the waiting piles up and spreads upstream.

Q The breaker is HALF-OPEN and one trial call fails. What happens?

It goes straight back to OPEN and the cool-down timer restarts. Some libraries also make the next cool-down longer.

Q You have 20 pods, each with its own breaker. Is that a problem?

Usually it's fine. Each pod learns on its own, so they trip at slightly different times and some keep sending traffic for a while. If you need one shared view, do it at the mesh or gateway level (outlier detection) instead of sharing state between pods.

Q Should a 404 count as a failure?

No. A 404 means the request was wrong or the item doesn't exist, and the dependency is healthy. Count timeouts, connection errors, 5xx and 429.

remember!!

Fail fast, fall back, find out, then try again carefully. A breaker protects the caller *and* gives the dependency room to heal.

Further reading

- Michael T. Nygard, *Release It!*, 2nd ed., chapter "Stability Patterns".
- Martin Fowler, "CircuitBreaker" (martinfowler.com/bliki/CircuitBreaker.html).
- Microsoft Azure Architecture Center, "Circuit Breaker pattern".
- Resilience4j docs, "CircuitBreaker" (sliding windows, slow-call thresholds).

a.k.a. "there is no global undo button"

Saga

TL;DR

When one business action (like "place an order") touches several services, each with its own database, split it into a **sequence of local transactions**. Each step commits in its own service. If a later step fails, run **compensating actions** that undo the earlier steps, in reverse order.

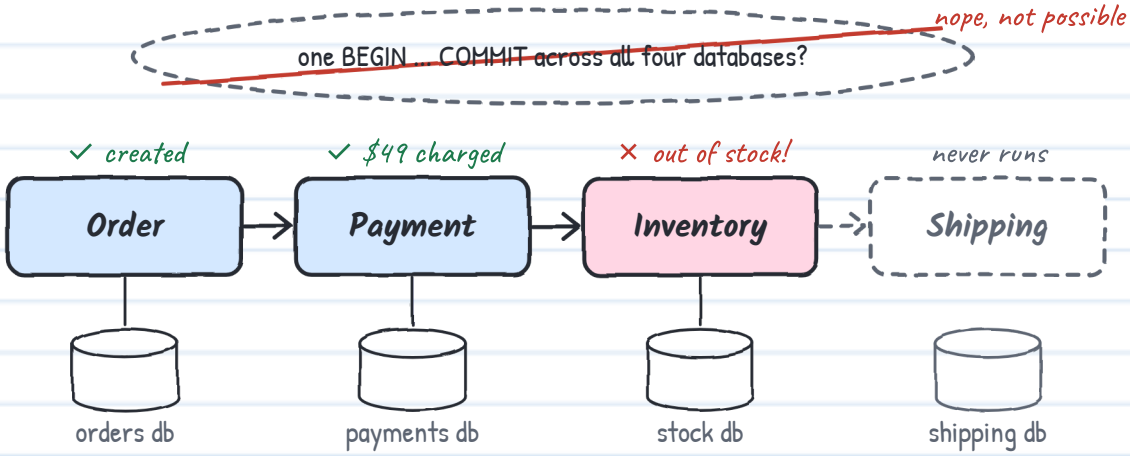
Family: *data management / consistency* | **Two styles:** *choreography & orchestration*

First described: *Garcia-Molina & Salem, 1987*

Pairs with: *outbox, idempotency, event-driven messaging*

1 The problem *why can't we just use a transaction?*

In a monolith, "place order" is one database transaction: `BEGIN`, do everything, `COMMIT`. If anything fails, `ROLLBACK` and it's as if nothing happened. In microservices each service owns its own database, so that safety net is gone.



the customer paid $49 for nothing, and nothing rolls it back

▲ *Each service committed its own part. When step 3 fails, steps 1 and 2 are already permanent.*

So the questions are: how do we keep the data **consistent** across services, and what do we do with the steps that already succeeded?

Why not two-phase commit (2PC)?

2PC (XA transactions) *can* make several databases commit together, but it's a poor fit for microservices:

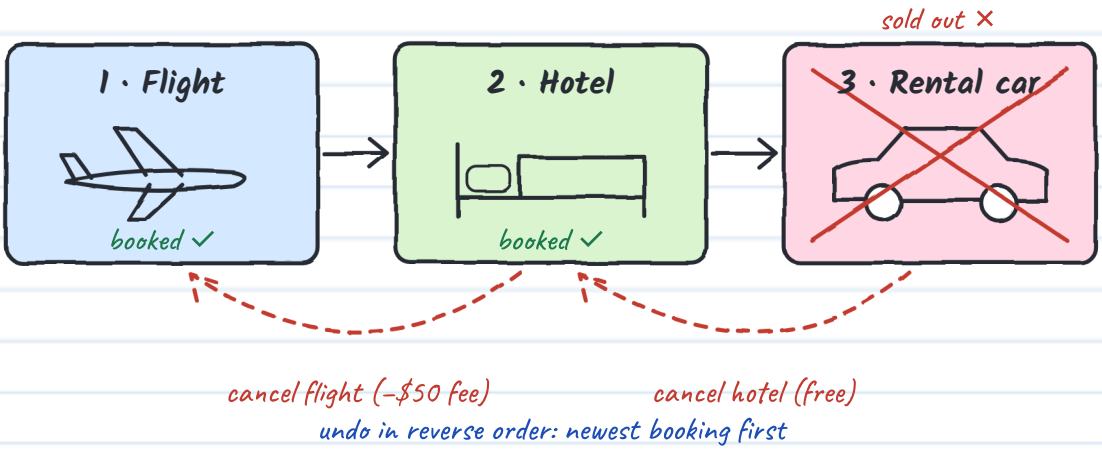
- Every participant **holds locks** until the coordinator says "commit". Slow participant = everyone waits.
- The coordinator is a **single point of failure**. If it dies mid-way, participants can be stuck "in doubt".
- Availability drops: the transaction succeeds only if *every* service is up at the same moment.
- Many modern stores and brokers (Kafka, most NoSQL databases, SaaS APIs) don't support it at all.

> ↳ *CAP reminder: across a network you pick availability OR strong consistency during a partition. Sagas pick availability + eventual consistency.*

2 Real-life analogy *booking a trip*

You're planning a holiday. You book a **flight** with an airline, a **hotel** on a booking site and a **rental car** from a car company. Three different companies, and there's no single button that says "book all three or none".

- Flight booked ✓, hotel booked ✓, then the car company says **sold out**.
- No car means the trip doesn't work. So you **cancel the hotel**, then **cancel the flight**, newest first.
- The flight cancellation costs a **$50 fee**. Cancelling isn't the same as it never happening. That's the big difference between a compensation and a database rollback.
- Smart travellers book the hard-to-get, non-refundable thing **last**, after everything else is confirmed.



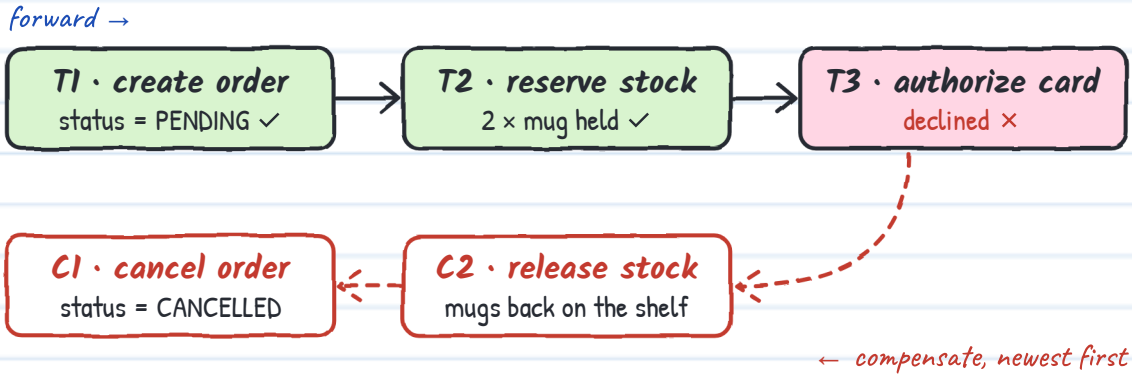
▲ *Each booking is its own "transaction". Cancelling is a new action with its own cost, not a rewind.*

Translating the trip to software

The trip	The saga
Each booking with a different company →	A **local transaction** in one service, committed on its own
Cancelling a booking →	A **compensating transaction** (a semantic undo)
The $50 cancellation fee →	Compensation isn't a perfect rollback; side effects can remain
A travel agent who books everything for you →	**Orchestration**: one coordinator tells each service what to do
A group chat where each friend books after the last one posts "done!" →	**Choreography**: services react to each other's events
Booking the non-refundable thing last →	The **pivot** step: once it succeeds, the saga only moves forward
Your hotel shows "pending" until the card clears →	A **semantic lock**: a PENDING status others can see

3 How it solves it *steps + undo steps*

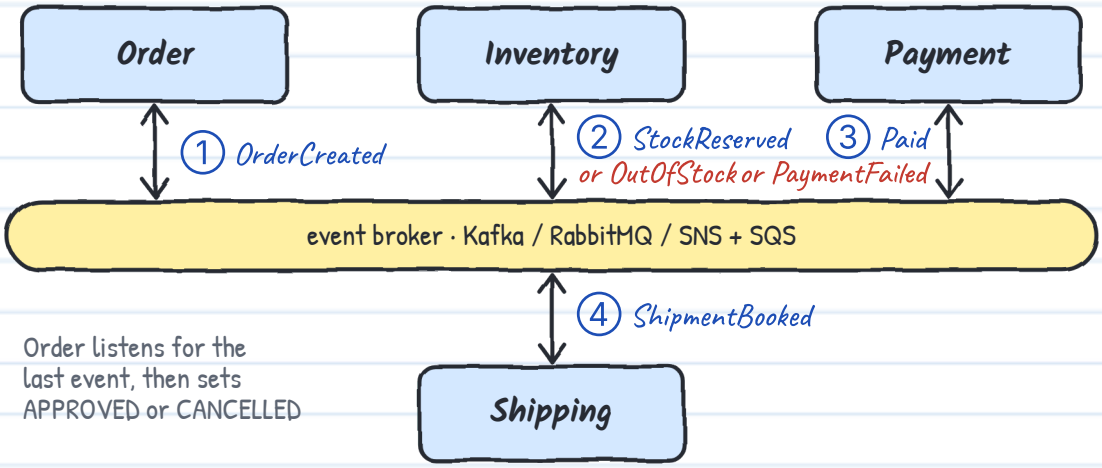
For each step T_i you design a matching compensation C_i. Run T_1, T_2, T_3… If T_3 fails, run C_2 then C_1. Every step commits right away in its own database, so **no locks are held across services.**



▲ *The order never "disappears". It ends as CANCELLED, which is a real, visible state.*

Style A · Choreography *(no boss, everyone listens)*

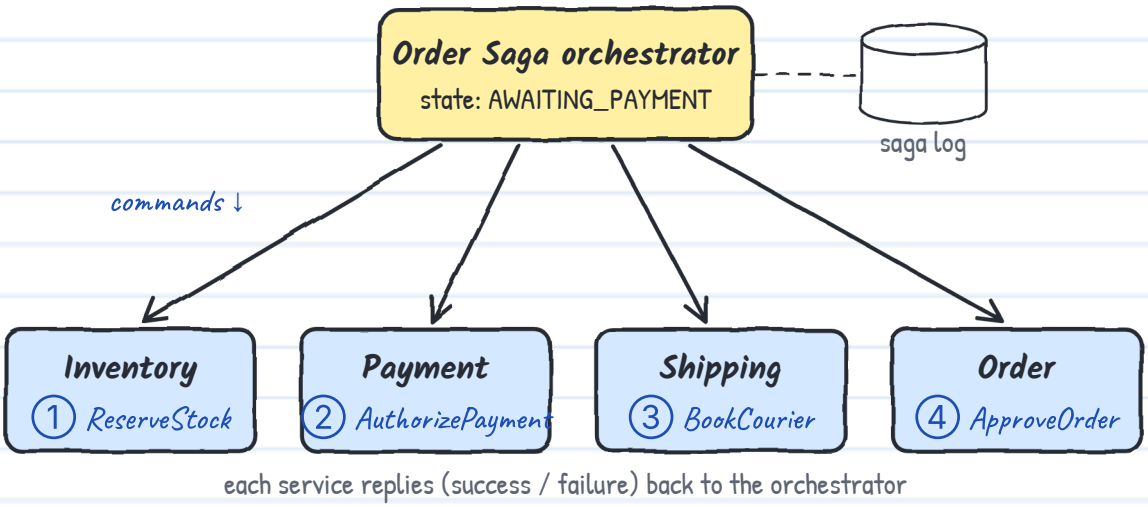
There's no central coordinator. Each service does its step, then **publishes an event**. The next service is subscribed to that event and reacts. Failures are events too (`PaymentFailed`), and the services that already acted listen for them and compensate.



▲ *Loose coupling and no single point of failure, but the full flow isn't written down anywhere.*

Style B · Orchestration *(one coordinator runs the show)*

A dedicated **saga orchestrator** knows the whole recipe. It sends a **command** to each service ("reserve stock for order 42"), waits for the reply, records progress in a **saga log**, and decides the next step or which compensations to run.



▲ *Easy to read, test and monitor in one place. The orchestrator must itself be durable and highly available.*

Pick choreography when...

The flow is short (2–4 steps), linear, and services are owned by different teams who want to stay loosely coupled.

Pick orchestration when...

The flow is long, has branches or timeouts, or the business wants to see "where is order 42 stuck?" in one place.

Designing the steps: our order saga

Order the steps so the ones most likely to fail and easiest to undo come first.

Step (T_i)	Compensation (C_i) / type
1. Order: create order with status **PENDING** →	Set status **CANCELLED** · compensatable
2. Inventory: reserve 2 × mug →	Release the reservation · compensatable
3. Payment: charge $49 →	**Pivot**: point of no return. If it succeeds, we only go forward
4. Shipping: book the courier →	Retry until it works · retriable
5. Order: set **APPROVED**, send confirmation email →	Retry until it works · retriable

> ↳ *rule of thumb from the lecture: compensatable steps → pivot → retriable steps. Never put a step you can't undo before one that might still fail.*

Vocabulary for the exam

Local transaction

An ACID transaction inside one service's own database.

Compensating transaction

A new action that semantically undoes an earlier step (refund, release, cancel).

Pivot transaction

The go/no-go step. After it commits, the saga must run to completion.

Retriable transaction

A step after the pivot that is guaranteed to succeed eventually if you keep retrying.

Semantic lock

A flag like PENDING that tells everyone else "this record is mid-saga".

Transactional outbox

Write the event to an outbox table in the same DB transaction, then publish it, so the update and the event can't drift apart.

run-saga.ts *orchestration, simplified*

```ts
type Step<C> = {
  name: string;
  run: (ctx: C) => Promise<void>;           // a local transaction in ONE service
  compensate?: (ctx: C) => Promise<void>;   // semantic undo, must be idempotent
  pivot?: true;                              // after this, never go back
};

export async function runSaga<C>(sagaId: string, steps: Step<C>[], ctx: C) {
  const done: Step<C>[] = [];
  let pastPivot = false;
  for (const step of steps) {
    try {
      await withRetry(() => step.run(ctx));            // idempotency key = sagaId + step
      await sagaLog.save(sagaId, step.name, "DONE");   // survive a crash, resume later
      done.push(step);
      if (step.pivot) pastPivot = true;
    } catch (err) {
      if (pastPivot) { await alertHuman(sagaId, step.name, err); throw err; } // can't undo now
      await sagaLog.save(sagaId, step.name, "COMPENSATING");
      for (const s of [...done].reverse()) {             // newest first
        if (s.compensate) await withRetry(() => s.compensate!(ctx));
      }
      throw new SagaAborted(sagaId, step.name, err);
    }
  }
}

await runSaga(order.id, [
  { name: "createOrder",  run: c => orders.create(c, "PENDING"), compensate: c => orders.cancel(c) },
  { name: "reserveStock", run: c => inventory.reserve(c),        compensate: c => inventory.release(c) },
  { name: "chargeCard",   run: c => payments.charge(c), pivot: true },
  { name: "bookCourier",  run: c => shipping.book(c) },
  { name: "approveOrder", run: c => orders.approve(c) },
], ctx);
```

↳ *real tools so we don't build the log/retry/resume ourselves:*
Temporal, AWS Step Functions, Azure Durable Functions, Camunda

> / Zeebe, MassTransit sagas (.NET), Axon / Eventuate Tram (Java).

4 Gains vs losses *nothing is free*

What we gain

+ **Consistency across services** without distributed locks or 2PC.
+ **Each service keeps its own database** and can be deployed and scaled on its own.
+ **High availability.** No coordinator holding locks; a slow service delays one saga, not everyone.
+ **Long-running processes** are fine: a step can wait minutes or days (e.g. manual approval).
+ **Failure handling is explicit** and written in business terms (refund, release, apologise).

What we lose

− **Only eventual consistency.** Others can see in-between states like PENDING.
− **No isolation** (the "I" in ACID). Concurrent sagas can cause lost updates or dirty reads unless you add countermeasures.
− **Much more code.** Every step needs a compensation, and both must be idempotent and retryable.
− **Some things can't be undone.** A sent email or shipped parcel needs a semantic fix, like an apology email or a return label.
− **Harder to debug.** One business action spans many logs; you need correlation IDs and tracing.

The "no isolation" gotcha and its countermeasures

- **Semantic lock:** mark records PENDING so other requests know to wait, or treat them carefully.
- **Commutative updates:** design changes so the order doesn't matter (add/subtract, not "set to").
- **Pessimistic ordering:** move risky steps earlier so dirty data is less likely to be read.
- **Re-read before acting:** check the value hasn't changed (version number) before overwriting it.

5 Architecture ratings *1 = poor · 5 = great*

Same nine characteristics as the architecture styles. Baseline here: a microservices system that tries to keep cross-service data consistent with **distributed 2PC**, compared with the same system using **sagas**.

Characteristic	With the pattern	Before → after	Why
Simplicity *easy to understand?*		3 → 2 ▼ −1	Every step needs an undo, and you must reason about in-between states.
Low cost *cheap to build & run?*		3 → 3 = no change	More code and a broker or workflow engine, but no XA-capable infrastructure.
Deployability *safe, easy releases?*		2 → 4 ▲ +2	Services stay independent. Changing the event or command contract still needs care.
Testability *test with confidence?*		3 → 2 ▼ −1	Many failure paths: each step failing, each compensation failing, duplicates, timeouts.
Scalability *more users & data?*		2 → 5 ▲ +3	No global locks, so each service scales on its own.
Elasticity *sudden spikes?*		2 → 4 ▲ +2	Async steps can queue up during a spike and drain afterwards.
Fault tolerance *failure stays contained?*		2 → 4 ▲ +2	No blocking coordinator. Failures become explicit, recoverable states.
Performance *fast typical request?*		2 → 4 ▲ +2	No lock waiting. The user gets "order received" fast, while the full result is eventual.
Evolvability *easy to change?*		2 → 4 ▲ +2	Add a step or service without touching the others (orchestration makes this easier).

kept *gained* *lost* *ratings are relative guidance for a typical system, not laws*

6 Common pitfalls *copy these into the exam cheat-sheet*

✕ Compensations that aren't idempotent

The broker delivers "refund order 42" twice, and the customer is refunded twice.

Fix → idempotency keys (sagaId + step) and a "processed messages" table.

✕ The dual-write trap

The service updates its database, then crashes before publishing the event. The saga is stuck forever. ***Fix*** → transactional outbox (or change-data-capture) so the update and the event commit together.

✕ Exposing in-between states

The "Your order is confirmed!" email goes out at step 1, then payment fails at step 3. ***Fix*** → semantic lock (PENDING) and send confirmations only in a final, retriable step.

Thinking compensation = rollback

You can't un-send an email or un-ship a parcel. **Fix** → design semantic undos (apology email, return label) and put irreversible steps after the pivot.

No timeouts on steps

A service never replies and the saga waits forever, with stock reserved for nobody. **Fix** → a deadline per step, then compensate or escalate, and a dashboard of "stuck sagas".

Choreography spaghetti

Eight services reacting to each other's events; nobody can explain the full flow and events start going in circles. **Fix** → switch to orchestration once the flow passes about 4 steps or has branches.

A saga where a transaction would do

The data actually lives in one database, but the team builds a saga anyway. **Fix** → use a normal local ACID transaction. Sagas are for crossing service boundaries.

7 When to use it (and when not)

Use it when...

- [x] One business action changes data in **several services** with separate databases.
- [x] The business can live with a short "pending" period (eventual consistency).
- [x] The process is long-running: payments, bookings, onboarding, approvals.
- [x] Every step has a sensible undo, or can be placed after the pivot.

Skip it when...

- [ ] All the data is in one database. Use a local transaction.
- [ ] You need strict, immediate consistency (e.g. a ledger that must never show a wrong balance). Keep that data together.
- [ ] Steps can't be compensated and can't be reordered behind a pivot.
- [ ] The "services" are really one tightly coupled thing; maybe merge them first.

Related patterns

Transactional outbox · reliable events

Idempotent consumer · safe duplicates

Event sourcing · state as a log of events

CQRS · separate read models

Process manager / workflow engine

8 Self-check quiz

Q What's the difference between a rollback and a compensation? ✓

A rollback discards uncommitted changes, as if they never happened. A compensation is a new, committed action that semantically reverses an earlier committed one (a refund, a cancellation). Its effects, like a fee or an email, can stay visible.

Q Which step should be the pivot, and why? ✓

The step that is hardest or impossible to undo (often capturing payment). Put compensatable steps before it and steps that are certain to succeed with retries after it, so you never need to undo the pivot.

Q 6 steps with a branch for manual review: choreography or orchestration? ✓

Orchestration. The flow is long and branchy, and the orchestrator keeps it readable, testable and observable in one place.

Q Why do sagas need idempotency? ✓

Messages can be delivered more than once and steps get retried after timeouts. Without idempotency, a step or compensation might run twice (double refund, double reservation).

remember!!

There's no global undo button. Every step needs its own "oops" button, and pressing it twice must be safe.

Further reading

- Hector Garcia-Molina & Kenneth Salem, "Sagas", ACM SIGMOD 1987.
- Chris Richardson, *Microservices Patterns*, chapter 4 "Managing transactions with sagas".
- microservices.io, "Pattern: Saga" and "Pattern: Transactional outbox".
- Microsoft Azure Architecture Center, "Saga distributed transactions pattern".